

EECS 4080 – Supervisor Feedback

Milestone 3: DistilBERT Training & Fine-Tuning & Midterm Progress Report

Reviewed via written submission, Midterm Progress Report, and GitHub repository

Overall Assessment

This is your strongest submission to date. The Milestone 3 training work is technically sophisticated, well-reasoned, and fully verifiable against your GitHub repository. All three engineering strategies are correctly implemented in code, the results JSONs match the submitted numbers exactly, and your decision to select the optimal epoch by lowest FPR rather than highest F1 reflects genuine understanding of the security context. The Midterm Progress Report is accurate and clearly written.

Three items require your attention before Milestone 4: a missing inference latency metric (a contract-required figure), an unacknowledged gap between your F1 score and the contract target, and the need to strengthen your Midterm Progress Report with a forward-looking plan for the remaining milestones.

What Went Well

Milestone 3 – DistilBERT Training

- **Engineering strategies grounded in observed evidence.** Each of your three strategies directly addresses a specific failure mode identified in Milestone 2b — this is exactly the kind of iterative, evidence-driven reasoning that belongs in a research project.
- **Left-side truncation — correctly implemented.** `truncation_side='left'` is confirmed in `src/train_custom_model.py` at line 71. The rationale (preserving the malicious payload at the end of long prompts) is sound and well-explained.
- **Asymmetric loss function — mathematically rigorous.** Your walkthrough of `CrossEntropyLoss`, the derivation of the penalty equations, and the connection to FPR reduction is the strongest piece of technical writing you have produced. The implementation in `WeightedTrainer` is clean and correct.
- **Automated epoch selection by FPR — shows research maturity.** Using `metric_for_best_model='fpr'` with `greater_is_better=False` to automatically discard the overfitted Epoch 3 and retrieve Epoch 2 is a principled solution to both the overfitting problem and evaluation integrity.
- **Results fully verifiable.** All numbers in the written submission — validation logs, test set metrics, and confusion matrix counts — match the committed JSON files in `results/` exactly. This is reproducible science.
- **FPR target met on the blind test set.** 4.05% FPR on 1,104 blind test prompts, against a contract ceiling of < 5%. This is confirmed and passes.
- **Model weights hosted on Hugging Face Hub.** A practical and professional workaround for GitHub's file size limits. The README link to [thienyu/prompt-injection-guardrail](#) is the right approach.

Midterm Progress Report

- **Accurate summary of all three milestones.** Every figure in the progress report is consistent with the committed repository results. Nothing has been embellished.
- **Clear causal explanation of the baseline limitations.** Your discussion of the jailbreak dataset composition and the 512-token bottleneck is well-contextualized and grounded in the actual EDA findings.

- **Comparative performance matrix.** The side-by-side table of baseline vs. custom DistilBERT results is effective and appropriate for a progress report audience.

Performance Summary — Blind Test Set (1,104 Prompts)

Metric	Reported Value	Contract Target	Status
F1-Score	83.31%	$\geq 85\%$	Below Target
False Positive Rate (FPR)	4.05%	$< 5\%$	Passes
Precision	92.48%	—	Informational
Recall	75.80%	—	Informational
Accuracy	87.95%	—	Informational
Inference Latency	Not reported	< 50 ms/prompt	Missing

The contract's Baseline Comparison clause is satisfied: your custom DistilBERT (83.31% F1) exceeds the ProtectAI baseline (66.05% F1) by over 17 percentage points — well above the $\pm 5\%$ threshold. This clause passes.

Gaps and Required Actions

1. F1 Score Below Contract Target — Acknowledge and Contextualize

Your contract specifies an $F1 \geq 85\%$ for the primary DistilBERT model. Your test set result of 83.31% is 1.69 percentage points below this threshold. This is not a large gap, and there is a defensible explanation — your dataset is dominated by hard jailbreak-style adversarial examples that even the industry-standard DeBERTa struggles with. However, neither your Milestone 3 document nor your Midterm Progress Report acknowledges this shortfall.

You do not need to re-train the model — but you should:

- Add a brief statement in your written analysis (and in the Midterm Progress Report if you revise it) noting that 83.31% falls slightly below the 85% F1 target.
- Explain why: the training corpus is 88% jailbreak-style adversarial data (rogue-security/prompt-injections-benchmark), which is significantly harder than the Deepset-only distribution the contract target was calibrated against.
- Note whether you plan to attempt further F1 improvement in Milestone 4 or 5 (e.g., additional epochs with adjusted hyperparameters, or whether you accept this as the baseline for comparison).

2. Inference Latency — Required Metric Not Reported

The contract explicitly requires mean inference latency < 50 ms per prompt. This is a named contract metric, not a nice-to-have. It is absent from your results JSON, your written submission, and your Midterm Progress Report.

The good news: the figure is derivable from your existing training logs without re-running anything. From your Epoch 2 validation results:

Value from Epoch 2 Logs	Derived Metric
eval_runtime: 19.3916s	~ 883 validation samples / 19.39s $\approx$ 21.9 ms per prompt

~ 22 ms per prompt is well within the < 50 ms target. Please add this to your work:

- Add `inference_latency_ms` to `results/custom_distilbert_metrics.json` (you can compute this precisely by timing a batch inference run with `time.perf_counter()`).

- Add one sentence to your Midterm Progress Report reporting the latency figure and confirming it passes the contract target.

3. `evaluate_custom_model.py` — Reproducibility Gap

Your evaluation script loads the model from the local path `models/custom_distilbert/`, but the model weights are hosted on Hugging Face Hub (not committed to the repository). This means a fresh clone of your repository cannot run the evaluation script without first downloading the weights manually. The README does not currently include this step.

Please add the following to your README's Environment Setup section:

- A command to download the model weights from Hugging Face before running `evaluate_custom_model.py`
- Alternatively, modify `evaluate_custom_model.py` to accept the Hugging Face Hub model ID directly so it downloads automatically on first run.

4. Midterm Progress Report — Add a Forward-Looking Section

Your progress report accurately summarizes everything completed so far, but it contains no plan for what comes next. A midterm report should include a brief section outlining the remaining milestones, any risks you anticipate, and your timeline confidence. The contract still has three significant deliverables outstanding:

- Milestone 4 (due July 5): Embedding Vector DB classifier and LLM-as-a-Judge implementation; automated benchmarking script.
- Milestone 5 (due July 19): Exhaustive benchmarking across all models; Out-of-Distribution (OOD) evaluation against indirect injections (BIPIA dataset); draft final report.
- OOD Generalization: The contract requires testing against an unseen indirect injection dataset (BIPIA) with a target OOD F1 ≥ 0.70 . This evaluation has not been mentioned in any submission to date. You should begin planning for it now, as it requires sourcing the BIPIA dataset and designing the OOD evaluation protocol.

Please revise the Midterm Progress Report to include a Plan Forward section covering these items before your next check-in.